MediClaim AI

Frontend MVP Documentation

Medical Document Assistant + Claim Risk Explainer

Item	Details
Frontend framework	React + TypeScript using Vite
Styling approach	Tailwind CSS with Apple-inspired minimal UI
UI libraries	Lucide React icons, Motion animations, Recharts ready for dashboard charts
Current status	Frontend MVP built and production build verified
Build check	npm run build completed successfully
Backend status	Not connected yet - UI uses localStorage demo data

Important project note

This frontend is an educational portfolio MVP. It uses localStorage and simulated responses to prove the user journey before connecting the ASP.NET Core Web API, database, document parser, and Semantic Kernel RAG services.

Prepared before backend development phase

Document Contents

1. Executive summary
2. Frontend purpose and scope
3. Technology stack
4. User workflow
5. Project structure
6. Route map
7. Task-by-task implementation log
8. Page documentation
9. Component documentation
10. State management and localStorage
11. Design system
12. Build and test checklist
13. Backend handoff plan
14. Interview-ready explanations

1. Executive Summary

MediClaim AI is a full-stack portfolio project designed to demonstrate healthcare document intelligence, claim risk review, and AI-assisted explanation workflows. This document describes the frontend MVP that has been completed before backend development starts.

The frontend is built using React, TypeScript, Vite, Tailwind CSS, Lucide icons, Motion animations, React Router, and browser localStorage. The current application simulates the user journey so the product flow can be validated before connecting real APIs.

Frontend capability	Current implementation
Landing page	Modern Apple-inspired public product page
Authentication UI	Login and register pages with demo token storage
Protected routes	Dashboard pages redirect unauthenticated users to login
Dashboard	Shows uploaded document count, claims reviewed, high-risk flags, recent documents, and recent risk findings
PDF upload page	Drag-and-drop PDF upload, file validation, progress animation, success state, localStorage document record
Claim CSV page	CSV upload, validation, simulated risk analysis, summary cards, risk table, explain modal
Document chat page	RAG-style chat UI with suggested questions, typing animation, source reference cards, active document selector
Final polish	Reusable AppShell, mobile navigation, reusable cards/buttons, 404 page, production build verification

2. Frontend Purpose and Scope

2.1 Purpose

The frontend acts as the user-facing layer of MediClaim AI. It allows users to navigate between document upload, claim review, dashboard metrics, and a RAG-style document chat interface.

2.2 Scope of current MVP

- Create a clean, modern SaaS-style interface suitable for a healthcare AI product.
- Build all core screens before backend integration.
- Simulate document uploads, claim analysis, and RAG answers using localStorage and sample data.
- Prepare routes, components, and data models so the ASP.NET Core backend can be connected later with minimal frontend redesign.

2.3 Out of scope for this frontend phase

- No real PDF text extraction yet.

- No real claim risk API yet.
- No real JWT authentication yet.
- No real Semantic Kernel or LLM connection yet.
- No database connection yet.

3. Technology Stack

Layer	Tool	Why it is used
Build tool	Vite	Fast React development server and production bundling
Frontend framework	React	Component-based UI for dashboard, forms, upload pages, and chat interface
Language	TypeScript	Adds type safety for components, props, routes, document records, and claim results
Styling	Tailwind CSS	Utility-first styling for fast, consistent Apple-inspired design
Icons	Lucide React	Clean line icons for navigation, dashboard cards, upload UI, risk states, and chat
Routing	React Router	Handles /, /login, /dashboard, /documents, /claims, /chat, and 404 routing
Animation	Motion	Adds subtle product-quality animation to landing and chat states
HTTP client	Axios	Installed for future API calls to ASP.NET Core backend
Charts	Recharts	Installed for future dashboard charts and analytics visualisation
Temporary storage	localStorage	Simulates backend persistence for documents, claim results, and demo token

4. User Workflow

The frontend is designed around a simple healthcare claim review journey. The current flow is fully testable without backend services.

Step	User action	Frontend behaviour
1	Open landing page	User sees product intro, feature cards, and Start demo button
2	Login or register	App stores a temporary demo token in

		localStorage
3	Open dashboard	Protected page shows metrics and quick actions
4	Upload PDF	Frontend validates file, shows progress, stores document metadata
5	Open chat	Uploaded PDF appears as active document for simulated RAG Q&A
6	Upload CSV	Frontend validates CSV, simulates analysis, shows risk results
7	Explain risk	User opens AI-style explanation modal for a selected claim
8	Return dashboard	Dashboard shows updated document and claim metrics

Landing -> Login/Register -> Dashboard -> Documents/Claims/Chat -> Dashboard Summary

5. Project Structure

The frontend is organised using a simple production-friendly structure. Pages represent routes, components hold reusable UI, types define shared data shapes, and utils contain temporary browser storage helpers.

```
frontend/
  src/
    App.tsx
    main.tsx
    index.css
    pages/
      LandingPage.tsx
      LoginPage.tsx
      RegisterPage.tsx
      DashboardPage.tsx
      DocumentUploadPage.tsx
      ClaimsUploadPage.tsx
      ChatPage.tsx
      NotFoundPage.tsx
    components/
      AppShell.tsx
      MobileNav.tsx
      ProtectedRoute.tsx
      StatCard.tsx
      RiskBadge.tsx
      ExplanationModal.tsx
      Button.tsx
      PageCard.tsx
    types/
```

```
    document.ts
    claim.ts
  utils/
    documentStore.ts
    claimStore.ts
```

Folder/file	Purpose
pages/	Each file maps to a visible screen or route
components/	Reusable UI blocks such as layout, badge, modal, protected route, cards, and navigation
types/	Shared TypeScript types for document metadata and claim results
utils/	localStorage helpers that simulate backend persistence
App.tsx	Central route configuration using React Router
index.css	Global theme, Tailwind import, background, font, scrollbar, and selection styling

6. Route Map

Route	Page component	Protected?	Purpose
/	LandingPage	No	Public product landing page
/login	LoginPage	No	Demo login flow
/register	RegisterPage	No	Demo registration flow
/dashboard	DashboardPage	Yes	Main workspace summary
/documents	DocumentUploadPage	Yes	PDF upload and document metadata store
/claims	ClaimsUploadPage	Yes	CSV upload, risk table, explanation modal
/chat	ChatPage	Yes	RAG-style document Q&A interface
*	NotFoundPage	No	Fallback for unknown routes

Route protection behaviour

ProtectedRoute checks whether localStorage contains a token. If no token exists, the user is redirected to /login. This will later be replaced by real JWT validation after ASP.NET Core authentication is implemented.

7. Task-by-Task Implementation Log

Task	What was added	Why it matters
1. UI foundation	React + TypeScript app, Tailwind, Lucide, Motion, first landing screen	Established design direction and project setup
2. Routing	Created pages and React Router configuration	Separated the app into real product screens
3. AppShell layout	Sidebar, header, mobile nav, consistent dashboard shell	Made internal pages feel like one SaaS product
4. PDF upload UI	Drag-and-drop, validation, selected file preview, progress animation, success state	Prepared the document upload workflow
5. CSV claim UI	CSV validation, sample results, risk badges, summary cards	Prepared claim risk review workflow
6. RAG chat UI	Chat bubbles, suggested questions, typing state, source cards	Prepared AI document assistant interface
7. UI polish	Reusable Button, PageCard, MobileNav, scrollbar, mobile responsiveness	Improved maintainability and professional feel
8. Login/Register	Auth screens and demo token storage	Prepared for JWT backend authentication
9. Protected routes	ProtectedRoute and logout flow	Added realistic app access control
10. Document store	localStorage document metadata shared across dashboard and chat	Connected upload flow to other screens
11. Claim store	localStorage claim results shared with dashboard	Connected claim analysis flow to metrics
12. Explanation modal	AI-style claim explanation modal	Added human-in-the-loop decision-support experience
13. Final polish	404 page and production build verification	Prepared frontend for GitHub and backend phase

8. Page Documentation

LandingPage.tsx

Public-facing landing screen

- Displays product name, hero message, feature cards, and demo entry point.
- Uses Motion for entry animations and Lucide icons for visual clarity.
- Start demo routes to login, while the page communicates the app value proposition.

LoginPage.tsx

Demo sign-in screen

- Collects email and password values using React state.
- Stores demo-token in localStorage when submitted.
- Redirects user to /dashboard using useNavigate.

RegisterPage.tsx

Demo registration screen

- Collects full name, email, and password.
- Stores demo user details and token in localStorage.
- Redirects to the dashboard after simulated registration.

DashboardPage.tsx

Main workspace summary

- Reads documents from documentStore and claims from claimStore.
- Shows metrics for uploaded documents, claims reviewed, and high-risk flags.
- Displays quick actions, recent documents, recent risk findings, and build progress.

DocumentUploadPage.tsx

Interactive PDF upload screen

- Uses useRef to trigger hidden file input.
- Validates file type and size.
- Supports drag-and-drop.
- Simulates upload progress and saves document metadata in localStorage.

ClaimsUploadPage.tsx

Interactive CSV claim review screen

- Validates CSV file type and size.
- Simulates analysis using sample claim results.
- Shows summary cards, risk badges, risk table, and explanation button.
- Saves claim results into localStorage for dashboard metrics.

ChatPage.tsx

RAG-style document Q&A screen

- Loads uploaded documents from localStorage.
- Allows active document selection.
- Shows suggested questions, chat bubbles, simulated assistant answers, typing state, and source cards.

NotFoundPage.tsx

404 fallback screen

- Handles invalid routes gracefully.

- Provides a clear link back to the dashboard.

9. Component Documentation

Component	Role	Key idea
AppShell	Main layout wrapper for protected pages	Provides sidebar, top header, mobile navigation, and consistent workspace styling
MobileNav	Mobile bottom navigation	Shows dashboard, documents, claims, and chat links on small screens
ProtectedRoute	Access control wrapper	Redirects user to /login when token is missing
StatCard	Dashboard metric card	Displays a label, value, and helper text
RiskBadge	Risk label	Maps Low, Medium, High to green, amber, and red visual states
ExplanationModal	Claim explanation popup	Shows risk summary, reason, and next review step for selected claim
Button	Reusable styled button	Defines consistent primary, dark, secondary, and danger variants
PageCard	Reusable content container	Provides shared card styling for future pages

10. State Management and localStorage

The current frontend uses React state for screen-level interactions and localStorage for temporary persistence across pages. This is intentionally used only for the MVP before backend integration.

State/data	Where used	Purpose
token	Login/Register/ProtectedRoute/AppShell logout	Simulates logged-in session
mediclaim_documents	DocumentUploadPage, DashboardPage, ChatPage	Stores uploaded PDF metadata
mediclaim_claim_results	ClaimsUploadPage, DashboardPage	Stores latest claim analysis results
selectedFile	DocumentUploadPage, ClaimsUploadPage	Tracks selected upload file
status	Upload pages	Tracks idle, uploading/analysing, success, or error state
progress	Upload pages	Shows frontend progress animation
selectedClaim	ClaimsUploadPage	Controls the explanation modal

messages	ChatPage	Stores user and assistant chat bubbles

10.1 documentStore.ts explanation

- getDemoDocuments reads document records from localStorage and safely returns an empty array if no data exists.
- saveDemoDocument creates a new document metadata object containing ID, file name, file size, upload time, and simulated chunk count.
- Duplicate prevention checks file name and file size so the same uploaded PDF is not saved multiple times.
- clearDemoDocuments removes the demo document store for reset/testing purposes.

10.2 claimStore.ts explanation

- getDemoClaims reads stored claim analysis results.
- saveDemoClaims saves the latest simulated claim risk output.
- clearDemoClaims removes saved claim results during testing or reset.

11. Design System

The UI follows a modern Apple-inspired visual direction: soft grey backgrounds, rounded cards, subtle shadows, dark primary actions, blue accent actions, and clean spacing.

Design element	Implementation
Primary blue	#0071E3 for main actions and active highlights
Dark text/action	#1D1D1F for Apple-style dark text and dark buttons
Background	#F5F5F7 and radial gradients for soft neutral surface
Cards	Large border radius, white/transparent backgrounds, light borders, soft shadows
Buttons	Rounded full, comfortable padding, hover scale, active scale
Typography	System font stack: -apple-system, BlinkMacSystemFont, SF Pro Display, Inter, Segoe UI
Risk colours	Green for Low, amber for Medium, red for High
Animation	Subtle Motion entry animation, hover lift, progress bars, typing spinner

12. Build and Test Checklist

Test area	Expected result	Status
npm run build	Production build completes without TypeScript errors	Passed
Landing page	Loads product hero and feature cards	Passed
Login/Register	Stores demo token and redirects to dashboard	Passed
Protected routes	Redirect to /login when token is missing	Passed
Logout	Clears token and returns user to /login	Passed
PDF upload	Validates PDF, shows progress, saves metadata once	Passed
Duplicate PDF prevention	Same PDF does not appear twice after repeated upload	Passed
Dashboard document count	Reflects stored PDF uploads	Passed
CSV upload	Validates CSV and shows sample risk results	Passed
Dashboard claim count	Reflects stored claim results	Passed
Explanation modal	Opens and closes correctly for selected claim	Passed
Chat page	Shows active uploaded document and simulated RAG answer	Passed
404 page	Invalid route shows friendly page not found screen	Passed

Build verification command:
npm run build

Successful result observed:
- TypeScript build completed
- Vite production bundle created
- No blocking frontend errors

13. Backend Handoff Plan

The frontend is intentionally prepared for backend integration. The following mapping shows how the current demo functions will later connect to ASP.NET Core Web API endpoints.

Frontend feature	Current behaviour	Future backend endpoint
Login	Stores demo token	POST /api/auth/login

Register	Stores demo token and user details	POST /api/auth/register
PDF upload	Saves document metadata to localStorage	POST /api/documents/upload
Document list	Reads from localStorage	GET /api/documents
RAG chat	Simulated answer and source cards	POST /api/chat/ask
Claim CSV upload	Simulated sample risk results	POST /api/claims/upload
Claim explanation	Frontend-generated modal text	POST /api/claims/explain
Dashboard summary	Reads localStorage counts	GET /api/dashboard/summary

13.1 What will change after backend starts

- localStorage document and claim stores will be replaced by API calls.
- Demo token will be replaced with real JWT returned by ASP.NET Core authentication.
- PDF upload will send FormData to the backend and receive document ID, chunk count, and preview text.
- Claim CSV upload will send the file to the backend and receive real rule-based risk results.
- Chat page will send document ID and question to the backend RAG service.
- Explanation modal will call the backend AI explanation endpoint for selected risk findings.

14. Interview-Ready Explanations

Question	Strong answer
Why did you build the UI before the backend?	I wanted to validate the user journey first. Building the frontend early helped define required API endpoints, data shapes, and workflows before backend implementation.
Why React and TypeScript?	React fits dashboard and interactive UI development well, while TypeScript helps keep API response shapes, props, and shared types safer and easier to maintain.
Why use localStorage in the MVP?	It lets me simulate persistence across pages before the database and API exist. It is temporary and will be replaced with backend endpoints.
How did you design the claim risk UI?	I used clear Low, Medium, and High badges and an explanation modal so the app supports human review rather than making automatic fraud accusations.
How does the chat UI prepare for RAG?	The chat screen already supports active document selection, user questions, assistant responses, typing state, and source cards. The backend later only needs to replace the simulated answer with real retrieved context and LLM output.
What did you do to make it production-style?	I added protected routes, reusable components, mobile navigation, consistent layout, file validation, error states, 404

	page, and verified the production build.

15. Current Frontend Limitations

- Authentication is only simulated with localStorage.
- PDF content is not actually extracted in the frontend.
- Claim risk results are currently sample data, not calculated from the uploaded CSV contents.
- RAG answers are simulated and not generated from real document chunks.
- No real backend database exists yet.
- No API error handling has been connected yet.

16. Immediate Next Steps

15. Commit the frontend MVP to GitHub.
16. Create the ASP.NET Core Web API solution.
17. Add Swagger and a HealthController endpoint.
18. Build authentication endpoints and replace demo localStorage token with real JWT.
19. Build document upload endpoint and connect DocumentUploadPage to the API.
20. Build claim upload endpoint and connect ClaimsUploadPage to the API.
21. Add RAG endpoint using Semantic Kernel and connect ChatPage.
22. Add Docker and CI/CD after the full-stack flow is working locally.

Frontend phase conclusion

The frontend MVP is now complete enough to demonstrate the product idea, explain the user journey, and guide backend API development. The next phase should focus on ASP.NET Core backend endpoints, database models, authentication, PDF processing, claim rule evaluation, and Semantic Kernel RAG integration.